

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 14. Functions

Lecture: 162. Object & Attribute Functions - Built-in

Section 1: Lecture Summary

This lecture focused on built-in Python functions related to **objects and their attributes**, demonstrating how Python treats everything as an object. The instructor introduced a group of key functions including `type()`, `isinstance()`, `hasattr()`, `getattr()`, `id()`, `dir()`, and `repr()`, explaining their usage and purpose. Practical examples were run in Jupyter Notebook to show each function in action. The functions help in inspecting the type, checking membership in a class, verifying attributes, retrieving attributes or methods from objects or modules, getting the unique identity of objects, listing all members of objects or modules, and obtaining a string representation.

Section 2: Key Concepts and Explanations

- `type(object)`

Returns the type (class) of the given object. Useful for identifying the exact type such as integer, string, list, tuple, etc.

- `isinstance(object, classinfo)`

Checks if an object is an instance of a specified class or a tuple of classes. Returns `True` if yes; otherwise `False`.

- `hasattr(object, attribute_name)`

Determines if an object has a given attribute or method. Returns `True` if the attribute exists, else `False`.

- `getattr(object, attribute_name[, default])`

Retrieves the value or reference to a named attribute or method from an object. If the attribute is a method, you can call it afterward. Optionally provides a default if the attribute does not exist.

- `id(object)`

Returns the unique identity (memory address) of the object during its lifetime. Identical immutable objects (like small integers) may share the same ID due to optimization, while mutable objects like lists have distinct IDs even if their contents are identical.

- `dir(object=None)`

Lists all attributes and methods of an object or module, both built-in and user-defined. Useful to explore the capabilities of an object or module.

- repr(object)

Returns the official string representation of an object, showing the value as it would be input in code, often including quotes for strings. Different from ``print()`` which outputs a “friendly” string without surrounding quotes.

Section 3: Example Code and Use Cases

type() example

```
print(type(10))          # <class 'int'>
print(type(3.14))        # <class 'float'>
print(type("Hello"))     # <class 'str'>
print(type([1, 2, 3]))    # <class 'list'>
print(type((1, 2, 3)))    # <class 'tuple'>
print(type({1, 2, 3}))    # <class 'set'>
print(type({"a": 1}))     # <class 'dict'>
print(type(None))        # <class 'NoneType'>
```

Prints the types of various Python objects.

isinstance() example

```
x = 10
print(isinstance(x, int))      # True
print(isinstance(x, float))    # False
print(isinstance(x, (int, float))) # True
print(isinstance("Hello", (int, str))) # True
```

Checks whether ``x`` is an instance of certain types.

hasattr() example

```
text = "hello"
print(hasattr(text, 'lower'))  # True (string has lower method)
print(hasattr(text, 'search')) # False (string has no search method)
print(hasattr(text, 'find'))   # True (string has find method)
```

Checks if the string ``text`` has certain methods.

getattr() example

```
import math
print(getattr(math, 'pi'))      # 3.141592653589793
sqrt_func = getattr(math, 'sqrt')
print(sqrt_func(25))            # 5.0
```

Retrieves attributes and functions from the math module, then calls the sqrt function.

id() example

```
x = 10
y = 10
print(id(x))    # e.g., 4335083568
print(id(y))    # same as id(x) since integers are interned
l1 = [1, 2, 3]
l2 = [1, 2, 3]
print(id(l1))   # unique memory location
print(id(l2))   # different from l1
```

Shows that immutable integers may share IDs, but distinct lists do not.

dir() example

```
print(dir([1,2,3])) # lists all methods of list objects ['append',
'clear', 'copy', ...]
import math
print(dir(math))    # lists all members of math module ['acos', 'ceil',
'cos', ...]
```

Lists all attributes/methods of lists and the math module.

repr() example

```
text = "Hello World"
print(text)          # Hello World (no quotes)
print(repr(text))    # 'Hello World' (with quotes)
```

Shows difference between normal print and `repr` representation.

Section 4: Key Takeaways

- Everything in Python is an object, and these built-in functions help inspect and interact with these objects and their attributes.
- `type()` shows the exact class/type of an object.
- `isinstance()` verifies if an object belongs to one or multiple classes.
- `hasattr()` and `getattr()` help check for and retrieve attributes or methods dynamically.
- `id()` returns a unique identifier useful for understanding object identity and memory references.
- `dir()` provides a quick way to explore all members—methods and attributes—of an object or module.

- ``repr()`` outputs the official string representation of an object including all literal details (such as quotes for strings).

- Commonly used functions among these include ``type()``, ``isinstance()``, ``id()``, and ``dir()`` for everyday coding and debugging tasks.